2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) | 979-8-3315-9147-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ESEM64174.2025.00013

Rethinking Code Review Workflows with LLM Assistance: An Empirical Study

Fannar Steinn Aðalsteinsson*†, Björn Borgar Magnússon*†, Mislav Milicevic*,
Adam Nirving Davidsson*, Chih-Hong Cheng†‡
*WirelessCar Sweden AB, Gothenburg, Sweden
†Chalmers University of Technology, Gothenburg, Sweden
‡Carl von Ossietzky Universität Oldenburg, Oldenburg, Germany

***Abstract*—*Background*: Code reviews are a critical yet time-consuming aspect of modern software development, increasingly challenged by growing system complexity and the demand for faster delivery. *Aims*: We examine how large language models (LLMs) can support code reviews by addressing common inefficiencies and contextual gaps. *Method*: At WirelessCar Sweden AB, we conducted an exploratory field study to identify current challenges, followed by a field experiment with two LLM-assisted review prototypes: one providing upfront, AI-generated reviews and another enabling on-demand interaction. Both used a retrieval-augmented generation pipeline to assemble relevant contextual information. *Results*: The field study revealed frequent context switching, insufficient contextual information, and concerns around false positives. In practice, developers generally preferred the AI-led approach, especially for large or unfamiliar pull requests, though preferences varied with codebase familiarity and review risk. *Conclusions*: LLM-assisted reviews can reduce cognitive load and improve comprehension, with hybrid proactive/on-demand designs best balancing efficiency, trust, and reviewer control.**

***Index Terms*—Large Language Models; Code Review; Empirical Software Engineering**

Lay Abstract: Code reviews are a key practice for ensuring software quality, finding defects, and sharing knowledge among developers. Yet, they can be time-consuming and difficult, especially when changes are large, complex, or outside the reviewer's expertise. In collaboration with WirelessCar Sweden AB, we studied how Large Language Models (LLMs) could be integrated into code review workflows to reduce effort and improve understanding.

We began by interviewing developers to identify challenges in their current process. Common issues included delays in completing reviews, difficulty understanding large pull requests, frequent context switching, and insufficient background information. Developers saw potential in AI features such as summarizing changes, detecting hidden bugs, and checking compliance with requirements, but they also expressed concerns about false positives, missing context, and the need to trust AI-generated suggestions. These findings guided the design of two AI-assisted review tools: one that provided an upfront summary and flagged possible issues (AI-led Co-Reviewer) and another that only responded to reviewer questions (Interactive Assistant).

Multiple developers evaluated both tools during real code reviews. Most preferred the summary-first approach for large or unfamiliar changes, reporting time savings and improved comprehension. Our results show that AI can effectively support human reviewers when it is accurate, fast, context-aware, and integrated into existing tools. Such AI systems work best as trusted partners that complement, rather than replace, human judgment.

The first two authors contributed equally to this work.
Correspondence to: chih-hong.cheng@uni-oldenburg.de

I. Introduction

Code review (CR) is a cornerstone of modern software engineering, ensuring code quality, defect detection, and team knowledge sharing. Nevertheless, as software systems scale and development cycles accelerate, traditional manual code review practices struggle with inefficiencies, reviewer fatigue, and inconsistent outcomes. The rapid development of Large Language Models (LLMs) [1]–[3] has enabled AI to demonstrate strong capabilities across various software engineering tasks, including code generation and bug detection. While their integration into the coding activities has been smooth, the full potential of LLMs in code review remains underexplored.

Through a combination of observational research and field experiments conducted at WirelessCar Sweden AB[1] (where some but not all teams are permitted to utilize AI in development), this study investigates how LLMs can be meaningfully integrated into modern code review workflows to improve developer experience and potentially support review efficiency. The research is structured around two core questions: one diagnostic and one exploratory. The first question (RQ1) focuses on understanding current code review practices within the company and identifying opportunities for AI assistance, while the second question (RQ2) explores how developers perceive and interact with two variations of LLM-based tools during review tasks.

To address these research questions, we conducted a two-phase empirical study (Sec. IV) at WirelessCar Sweden AB.

- Phase 1 involved a field study through semi-structured interviews to uncover practical challenges in existing code review workflows and to identify potential areas where AI assistance could be beneficial. Importantly,

[1]https://www.wirelesscar.com/

979-8-3315-9147-2/25/$31.00 ©2025 IEEE
DOI 10.1109/ESEM64174.2025.00013

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio

based on these findings, we designed and implemented two variations of LLM-assisted review tools, one being an AI-led co-reviewer and the other an interactive assistant, integrated with retrieval-augmented generation (RAG) to provide contextual support.
- In Phase 2, we evaluated these tools in a real-world field experiment involving practicing developers. The results show that while the AI-led mode was generally preferred, especially for large or unfamiliar pull requests, preferences were context-dependent. Participants valued the tool's ability to provide faster understanding, improved thoroughness, and helpful contextual insights, though issues such as trust, false positives, and interface limitations were noted.

Altogether, our findings (Sec. V) suggest that LLMs can meaningfully augment, rather than replace, human reviewers, and highlight the importance of adaptive integration strategies that respect developers' workflow preferences and domain knowledge. Furthermore, our results point to clear design directions for future tools: AI assistance should be seamlessly embedded into existing developer environments (e.g., GitHub, IDEs, Slack), offer concise and structured feedback with minimal latency, and support both proactive and reactive interaction modes. For maximal utility, the assistant must be context-aware and capable of leveraging relevant project artifacts like code diffs, source files, and requirement tickets. Notably, participants also envisioned the assistant being valuable during the pre-review phase, helping authors improve pull requests before submission. These practical insights inform how LLMs can be responsibly and effectively integrated into real-world software development workflows.

II. Related Work

Since the emergence of ChatGPT, large language models have seen widespread adoption in all software engineering tasks, such as code generation, test case creation, or documentation. We refer readers to a review paper [4] published in 2024 for a summary of recently conducted research activities. In particular, recent works [5]–[10] have explored the application of AI, in specific LLMs, to automate or support code review tasks. The early result from Tufano *et al.* [10] presents a deep learning model trained to replicate reviewer-suggested code changes, aiming to partially automate the code review process. In the era of LLMs, the work from Tufano *et al.* [5] evaluates the measurable impact of AI-generated code reviews in controlled experiments over code with injected issues and code smells. It focuses on the performance of the LLM itself and the quantifiable effects on issue detection, time spent, and reviewer behavior. The work from Alami *et al.* [9] performs qualitative, interview-based exploration of developers' emotional and cognitive responses to AI-provided feedback compared to human-provided feedback in code reviews. The work from Vijayvergiya *et al.* [7] showcases the deployment and evaluation of a large-scale automated system that enforces coding standards and code smells using LLMs in code reviews. The work from Lin *et al.* [6] studies how to fine-tune LLM for better issue detection in code reviews. Similarly, the work from Rasheed *et al.* [8] develops a multi-agent LLM-based system for performing autonomous code reviews, focused on technical accuracy, issue detection, and actionable suggestions.

Summarizing the above results, one of the critical gaps is the study of the *preferred interaction* for an engineer assigned with the code-review task in collaboration with an LLM-enabled review assistant, which leads to the primary focus of our work (RQ2). We focus on understanding how LLMs can support, not replace, reviewers via practical integration strategies. We do not consider LLMs as human replacements mainly due to the fundamental concern that LLMs are still prone to hallucinations [11]. Another differentiator is the introduction of semi-structured interviews and thematic analysis to understand the real potential of LLMs in code reviews, reflecting the pain points of an organization developing complex software (RQ1). Understanding the pain points (RQ1) leads to the design of experiments and prototypical LLM-assisted code review tools to assess the preferred interaction (RQ2).

III. Research Questions

This study investigates how LLMs can be meaningfully integrated into modern code review workflows to improve developer experience and potentially support review efficiency. The research is structured around two core questions: one diagnostic and one exploratory. The first question focuses on understanding current code review practices and identifying opportunities for AI assistance, while the second question explores how developers perceive and interact with LLM-based tools during review tasks.

RQ1: *What practices, challenges, and expectations characterize modern code review processes, and where do developers see potential for AI-based assistance?*

This question aims to identify how code reviews are currently performed within the company, what challenges developers face, and how AI can be introduced in the code review process, what tasks it can effectively support, and the optimal balance between automation and human involvement.

RQ2: *How do developers perceive LLM-assisted code review tools, and what is the preferred interaction?*

This question explores the qualitative aspects of AI usage, including developer trust, satisfaction, and potential barriers to adoption, by considering different interaction modes. By answering this question, insights are provided into the usability, limitations, and acceptance of AI-assisted code reviews, helping to inform design decisions and future development of such tools.

IV. Methodology

Towards the above research questions, grounded in the literature review, we adopt a two-phase research design that is aimed at understanding both the current code review processes

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio

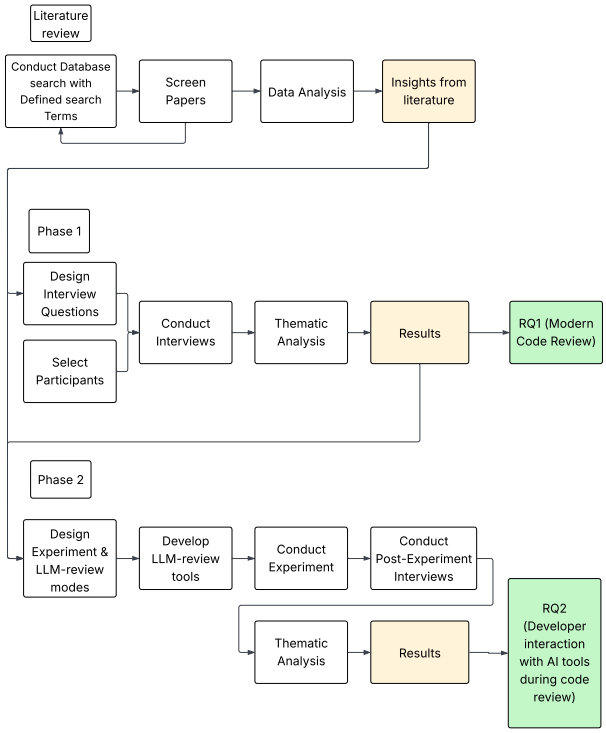

Fig. 1. Flowchart detailing the research workflow

in a natural environment and then intervening with an AI-based solution to observe the effects (see Fig. 1 for the research flowchart). Phase 1 involves an exploratory case study at WirelessCar to thoroughly understand existing manual code review practices, identify specific pain points, and potential AI application areas. The goal is to identify typical workflow steps, uncover key pain points, clarify why certain inefficiencies arise, how success is evaluated, capture the technical and organizational environment, and where AI might offer possible improvements. Phase 2 aims to assess how software developers at WirelessCar experience and evaluate the integration of a software artifact featuring LLM-assisted code review tools into their review process, thereby identifying design implications and best practices.

Importantly, rather than focusing on performance metrics or direct comparisons to existing tools as conducted in prior works, our emphasis on Phase 2 is on capturing qualitative insights into how developers interact with the AI assistant, what kinds of support they find valuable, and how such a tool might be ideally integrated into real-world code review practices. To make such an assessment possible, we created two variations of LLM-enabled review, namely the *AI-led mode (co-reviewer)* and the *interactive mode*. The design of the two modes is based on challenges and improvement opportunities identified during Phase 1 as well as insights collected from the literature.

Due to our research being qualitative in nature, in both phases, semi-structured interviews were conducted to receive feedback, and thematic analysis [12] has been applied to analyze the interview scripts being collected.

TABLE I
OVERVIEW OF INTERVIEW PARTICIPANTS IN PHASE 1

Participant ID	Role	Team
P1	Quality Assurance Specialist	Team A
P2	Application Developer	Team A
P3	Software Engineer	Team A
P4	Software Engineer	Team A
P5	Security Engineer	Team B
P6	Software Engineer	Team C
P7	Software Engineer	Teams D & B

A. Phase 1: Understanding the existing manual code review practices (field study)

In our field study, semi-structured interviews were conducted. Such a format balances consistency where all participants receive a core set of similar questions with flexibility where researchers can ask follow-up or clarifying questions as fits the flow of the interview [13]. The interview questions were developed around domains such as code review processes, challenges in code reviews, measuring code review success, and current and potential AI use cases to cover the previously mentioned objectives. The interviews were designed to fit comfortably within a 30-minute timeframe. However, in practice, the interviews ranged from 15 to 40 minutes each, with most lasting approximately half an hour.

We used *convenience sampling* [14] by interviewing people within the WirelessCar who were able and willing to participate. Participants were recruited via announcements on Slack channels and informal Slack messages that announced the study's purpose and form of the interview. Interviewees who handled different parts of the code and were reviewers at varying seniorities were sought. Additionally, some interviewees were from the same development team to gain multiple perspectives from the same team. Despite being limited in randomness, we believe our sampling still enables insights into a relatively wide range of review practices.

A total of seven participants were interviewed and are listed in Table I. The interviewees varied in gender and age, and the sample included software engineers, developers, security engineers, and quality assurance specialists across four development teams. Teams ranged in size from under five members to around sixteen members. The interviewees had different areas of expertise, with varying levels of experience and skill. The goal was to include individuals who could discuss both strategic and day-to-day review practices. Data saturation was reached after the seventh interview, as no new information or themes were emerging. This decision is grounded in the principle that qualitative data collection can be considered sufficient when additional interviews fail to yield new insights [15]. Data saturation is defined as the point at which no new themes are observed in the data, and studies show that although saturation often occurs within twelve interviews, the basic elements of major themes are frequently present as early as six [15]. The current sample

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restriction

is somewhat heterogeneous in terms of role and team, and the consistency of responses across participants suggested that the core aspects had been adequately captured. Although an eighth interview had been scheduled, the participant canceled, but given that saturation had already been achieved, it was decided not to reschedule the interview.

All interviews were held in English and conducted in the participant's active work environment, either on-site at WirelessCar's office or remotely via Microsoft Teams (two participants were interviewed remotely). This alignment with genuine working conditions is consistent with a field study approach since it allows developers to reference real pull requests, team communication channels, and examples of ongoing tasks. They could describe, for example, how a large refactoring pull request (PR) or an urgent bug fix impacted their review process in real-time. Conducting interviews in this natural setting helps reinforce the authenticity of participant responses and results in findings consistent with everyday workflows at WirelessCar.

B. Phase 2: Evaluating the LLM-enabled code review (field experiment)

1) Data Collection: To explore developer experience with the AI assistant during code reviews, two data collection methods were employed to capture user perspectives and behavioral interaction patterns. The primary data source consisted of post-interaction interviews with each participant. These were supported by a secondary data source consisting of researcher observation notes recorded during review sessions.

Participants were recruited again through convenience sampling, where individuals who participated in the earlier phase of the study were invited to return for Phase 2. However, only five were available and agreed to participate again. To reach a total of ten participants, five additional individuals were recruited via internal Slack channels, where the study's purpose and structure were briefly described. Of the ten participants, four belonged to the team responsible for the pull requests used in the experiment, while the remaining six were from other teams, allowing comparison across varying levels of codebase familiarity. Table II provides an overview of the participants, including their roles and team affiliations. The code used in the experiment originated from Team A, which is the team associated with the familiar participants.

2) Experiment Setup: The field experiment is designed to evaluate the developers' experiences with LLM-based code review assistance under two distinct interaction modes that were selected based on the findings from the Phase 1 field study. As indicated by the interview data in that earlier phase, developers most frequently identified a need for clearer up-front summaries of code changes as well as on-demand explanations for specific architectural or contextual details.

In each iteration of the experiment, a single participant engaged in a traditional code review scenario, conducted within their familiar development environment using their usual tools and platforms. In addition to these standard resources, the participant was provided access to our created LLM code review assistant as illustrated in Fig. 2. The task assigned to the participant was to perform two code reviews, each on a different pull request from two separate repositories within the company. The participant followed a different interaction mode with the AI assistant for each review. All sessions took place in conditions that mirrored each participant's normal working situation as closely as possible.

TABLE II
OVERVIEW OF EXPERIMENT AND INTERVIEW PARTICIPANTS IN PHASE 2

Participant ID	Role	Team
P1	Quality Assurance Specialist	Team A
P2	Application Developer	Team A
P3	Software Engineer	Team A
P5	Security Engineer	Team B
P7	Software Engineer	Teams D & B
P8	Software Engineer	Team E
P9	Software Engineer	Team A
P10	Software Engineer	Team E
P11	Software Engineer	Team F
P12	Software Engineer	Team G

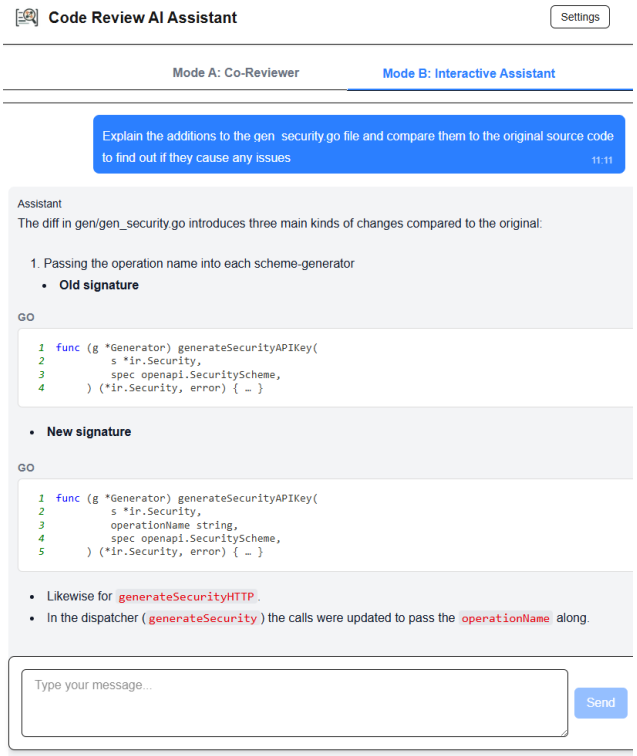

Fig. 2. Screenshot of the LLM-assisted code review interface in Mode B, reviewing a pull request from an open-source project available at `https://github.com/ogen-go/ogen/pull/1440`.

The two interaction modes are described below:

- **Mode A: Co-Reviewer** – In this mode, the AI assistant automatically generated a summary of the code under review, highlighting major changes and any potential points of interest or concern, before the reviewer started their own examination. The reviewer could then use this

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio

information to guide their review, optionally asking the AI-assistant follow-up questions. The participant could query the AI for clarification or more details about the summarized areas. This mode directly targeted the challenge previously identified as lacking immediate context, particularly for large or complex PRs.

- **Mode B: Interactive Assistant** – In this mode, the reviewer reviewed the code in their typical manner. The AI assistant did not proactively provide a summary or suggest issues upfront. Instead, the reviewer was free to consult the AI on demand, requesting clarifications about specific parts of the code or asking higher-level architectural questions. Reviewers were encouraged to ask targeted queries rather than requesting an overall summary of the changes. Here, the AI assistant only responded when explicitly prompted. This design choice not only reflected the Phase 1 feedback on needing a lightweight, on-demand tool but also addressed an identified challenge in related studies [5], where automatically highlighted lines can cause reviewers to miss other important areas. By making the AI passive, reviewers maintained their usual workflow and examined the entire codebase without unconsciously depending on the AI's initial hints.

To minimize review variability while still enabling comparison across interaction types, two specific pull requests of similar size and complexity were selected from the WirelessCar codebase. The selected pull requests each involved a moderate amount of change, requiring genuine reviewer effort without being excessively large or trivial. Before the first experiment session, a pilot run was conducted with two internal developers who were not part of the main participant pool. The pilot involved a walkthrough of the tool, the review task, and the interaction modes, followed by an informal test of the tool using the selected pull requests. The pilot was used to assess the suitability of the selected PRs, detect any major usability or technical issues in the tool, and evaluate whether the introduction and guidance provided to participants were sufficiently clear and effective. Each participant in the actual experiment then reviewed both PRs, using Mode A for one PR and Mode B for the other. The assignment of modes to PRs was rotated across participants to mitigate ordering effects. By having all participants conduct the same two code reviews with alternating modes, this approach allowed for a more controlled comparison of interaction styles while ensuring the tasks remained realistic and relevant to actual code review practices.

To investigate how varying levels of codebase familiarity may influence the use of the AI assistant, both developers belonging to the team that owns the selected PRs and developers from unrelated teams were invited to participate. Phase 1 interviews indicated that limited contextual knowledge can have negative effects on the review quality, and therefore, measuring differences in AI reliance across these two participant groups was expected to produce further insights.

Before starting the code review sessions, each participant was given a short onboarding briefing to explain the tool's

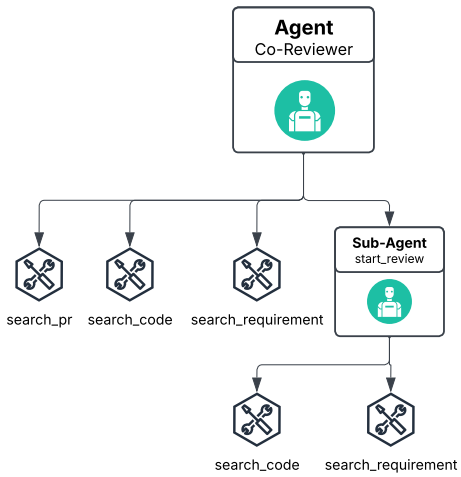

Fig. 3. Agentic tool structure in Co-Reviewer mode.

features as well as the structure and setup of both the study and the experiment. The two different interaction modes were introduced, along with guidance on how to effectively prompt the AI assistant to obtain useful and relevant responses. The participant then completed the two review sessions consecutively. While no in-depth feedback or direct assistance was provided during the sessions, limited guidance was offered when needed, for example, if participants inquired about specific ways of interacting with the assistant. Participants were also encouraged to think aloud during the experiment, often verbalizing their reasoning, confirming the AI's suggestions, or commenting on its usefulness. If the assistant's response did not meet expectations, participants were occasionally guided to try rephrasing or retrying the query. Researchers were present to observe the sessions, record notes, and perform the post-session data collection. Following the review sessions, short semi-structured interviews were conducted with the participant to reflect on their experience across the two modes and in comparison to their regular code review workflow.

3) Artifact Implementation: The software artifact developed for this study was a web-based chat interface designed to explore and evaluate different interaction styles in AI-assisted code reviews. Its primary purpose was to enable structured experimentation by allowing researchers to observe how developers interact with AI assistance in different contexts. The source code for the artifact is freely available (under GPLv3) on GitHub: frontend[2] and backend[3]. While not intended to be a production-level system, the artifact was designed to be realistic and usable enough to engage developers meaningfully. It supported live interaction through a chat interface, maintained session-specific context, and allowed researchers to configure session parameters such as interaction mode and PR data source.

The artifact consisted of a chat interface connected to a

[2]https://github.com/BearPays/code-review-assistant-ui
[3]https://github.com/BearPays/code-review-assistant-back

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio

backend system that integrated OpenAI's `o4-mini`[4] language model via API. The artifact was also supported by a Retrieval Augmented Generation (RAG) infrastructure built with LlamaIndex[5]. This setup enabled the AI assistant to produce more informed and context-aware responses by using project data such as code diffs, related source code files, and associated feature requirements (Jira tickets). The RAG index had to be manually prepared and indexed before experiments, ensuring complete control over the data available to the model in each experimental session. As highlighted by the literature and the Phase 1 interview results (see Sec. V-A2), lacking a broader repository context can lead to superficial AI feedback that overlooks critical design or architectural concerns. This RAG setup ensured that the LLM can reference deeper project-level information on demand. This setup not only enhanced the AI assistant's capacity to generate context-aware suggestions but also directly targeted the gap in existing tools and recent studies on the subject.

Internally, the assistant interacts with three core semantic tools:

- `search_pr`: accesses PR diffs and metadata.
- `search_code`: provides the full, unmodified source code of the repository.
- `search_requirements`: contains the feature requirement (the Jira ticket) motivating the PR.

In Mode A (Co-Reviewer), a fourth tool, `start_review`, was added. This tool contained a sub-agent that was designed to perform an initial, structured code review based on the full PR data and guided by a detailed review-specific prompt. Unlike the main agent, this sub-agent did not use `search_pr`, as all PR data was injected into its initial context via a prompt. This ensured that the agent considers everything in the PR data and examines each file change. By retrieving the PR data via a query engine, the agent might not consider all the data as required when generating a complete code review of all changes. The agentic structure for this setup is shown in Fig. 3.

V. Results

A. Results from Phase 1

Six themes emerged from the thematic analysis of the qualitative data and can be seen in Table III.

1) Observed Code Review Process: When reviewing the *informal review process at WirelessCar and their practices*, it appears to be similar to the typical asynchronous, tool-supported nature of modern code review (MCR) practices. However, the reliance on informal assignment and communication sometimes through Slack introduces variability, which might contrast with the structured practice of assigning a specific reviewer to the code patch commonly found in MCR practices. Additionally, WirelessCar places significant emphasis on contextual understanding and relies heavily on team members with domain expertise to assess the critical components. This contrasts with the broader responsibility-sharing model commonly found in large organizations that apply MCR.[6]

2) Common Challenges in Code Reviews: One of the most frequently mentioned challenges across the developer interviews was the issue of **delayed reviews**. Situations were described where PRs remained unreviewed for extended periods, often requiring repeated reminders for a reviewer to take action. One interviewee described it as:

> *"Sometimes you need to ping people more often, and sometimes the PR is very big, so people don't dare to pick it up"* *[P2]*

Several interviewees reported difficulties regarding reviewing **large or complex PRs**. PRs that combine new features, refactoring, and changes to infrastructure often become overwhelming. They note that this can result in more superficial reviews or longer delays.

Context switching was also noted as another major challenge. Developers mentioned that the cognitive burden of pausing ongoing work to review code, not at all related to their current work, to be challenging. This requires additional time to regain focus, and one interviewee highlighted the time lost cause of this:

> *"As soon as you need to context switch, even if it's just a three-minute thing, it's 20 minutes of lost time."* *[P7]*

Several interviewees explained that they sometimes **lack sufficient context when reviewing the code**. Sometimes, important details about why the change was made or its expected impact are missing from the PR description. This increases the time it takes for the reviewer to comprehend the PR and effectively point out defects or problems.

3) Current Use of AI in Software Development: The interviews revealed that common AI tools like GitHub Copilot[7] and ChatGPT[8] are commonly used for development tasks. Interviewees mentioned tasks like writing boilerplate code, assisting with syntax, and quickly generating documentation. One example mentioned was:

> *"[...] as to help to create the boilerplate stuff, it's outstanding, right? I mean, you do it in 30 seconds instead of a couple of hours. So I try to use it as much as possible during the development process."* *[P7]*

However, not all teams are allowed to use AI-generated code or share information (such as source code) with AI tools. Furthermore, the teams that are allowed to use AI tools only utilize them via an enterprise subscription, where the data

[4]https://platform.openai.com/docs/models/o4-mini
[5]https://www.llamaindex.ai/

[6]Due to space limits, quotes collected from the interview as well as detailed findings are not listed.
[7]https://github.com/features/copilot
[8]https://chatgpt.com/

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio

TABLE III
IDENTIFIED THEMES AND THEIR DESCRIPTIONS FROM ANALYSIS OF INTERVIEW DATA IN PHASE 1.

Theme	Description
Informal Review Process and Practices	Describes how teams coordinate and manage code reviews in practice, including informal communication, tool use, and the absence of structured processes or metrics.
Review Strategies and Evaluation Focus	Captures what developers focus on during the actual code review process.
Learning, Knowledge, and Review Expertise	Explores how code reviews serve as opportunities for learning and knowledge sharing within teams. It also captures the role of reviewer expertise in conducting effective reviews and the challenges that arise when reviewers lack sufficient understanding of the codebase or architecture.
Code Review Challenges	Identifies recurring challenges and inefficiencies encountered in the code review process.
Current AI adoption	Describes the current state of AI tool usage in development and code review.
Possible AI adoption in code reviews	Describes developers' expectations, suggestions, and concerns regarding future AI assistance in code reviews.

TABLE IV
IDENTIFIED THEMES AND THEIR DESCRIPTIONS FROM ANALYSIS OF DATA FROM PHASE 2.

Theme	Description
Accuracy, Reliability, and Trust	Focuses on the perceived correctness of AI-generated feedback, concerns about over-reliance, and varying levels of trust in the assistant's recommendations.
Efficiency and Thoroughness	Captures how the assistant affects review speed, cognitive load, issue detection, and the overall thoroughness of the code review process.
Integration Expectations and Limitations	Highlights developer expectations for seamless integration, responsive design, and context-aware suggestions, while also surfacing frustrations related to current UX and tooling limitations.
Usage Contexts and Interaction Patterns	Describes how interaction with the assistant varied based on review context, including preferences for different modes, alternative usage strategies, and team-specific practices.

provided is not used for training the models utilized by the tools. This ensures that no data is leaked from the organization via the use of AI tools.

All interviewees who use AI tools reported positive experiences in their development work. **None of the interviewees reported using AI tools as part of the formal code review process**. One interviewee did mention that some reviewers might occasionally paste code into tools like ChatGPT for clarification or explanation during reviews. However, beyond such informal use, AI has not been formally integrated into the code review workflow in any of the interviewed teams.

4) Potential AI Use Cases in Code Review: The interviewees expressed interest in potential AI integrations with the code review processes. They mentioned features like summarizing PR changes and accompanying descriptions where AI could generate a concise summary and help reviewers to quickly understand the intent of a code change. One interviewee mentioned:

> *"When you create the pull request, an AI bot could say, 'Hey, you're trying to achieve this—do you want this as your summary or description?"* *[P5]*

Interviewees also mention possibilities such as AI assisting in validating whether code meets stated requirements. Additionally, interviewees highlighted the potential for AI to detect hidden bugs or vulnerabilities, such as race conditions, dependency issues, or other subtle defects that human reviewers might overlook. As one developer put it:

> *"An AI would probably be able to identify a race condition, for instance, which, as I said, is an example that's borderline impossible to catch on the fly. In three minutes, you're never going to find that."* *[P7]*

Finally, some interviewees mentioned potential drawbacks of integrating AI into the code review process. These were mostly concerns about security risks and false positives, which could reduce the reviewer's trust in the AI assistant and divert their attention from real issues. As one interviewee put it:

> *"The problem with those kinds of checks is that, if they're not good enough, you stop reading them. We see that all the time, you get flooded with false positives, and then you miss the real issues because you start ignoring the feedback."* *[P7]*

B. Results from Phase 2

Table IV provides an overview of the themes along with descriptions of the themes.

1) Accuracy, Reliability, and Trust: Participants frequently commented on the accuracy of the assistant's feedback and how this influenced their trust in the tool. Several interviewees described the assistant as **generally accurate and capable of identifying relevant issues**. For example, one participant stated:

> *"As I see it, they were quite accurate [...] It was quite nice, not all of them, but a lot of them."* *[P9]*

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restriction

In multiple cases, the assistant's output was described as **confirming the developer's own thoughts or surfacing something they might not have otherwise caught**. Observation notes also reflect this, with one session noting that "the user said the AI caught exactly what he was looking for in a certain file." In another session, the reviewer remarked that "the summary gave something that he would not have seen."

Several participants also reported instances where the assistant produced **incorrect or unclear suggestions**. One participant questioned whether the tool was even "doing what it was asked," while another described the assistant incorrectly flagging a missing import. One participant noted:

> *"Sometimes it says some slightly strange things."* [P8]

When asked about concerns, participants expressed different perspectives. Several participants warned of the risk of over-relying on the assistant, especially in Mode A, where the assistant led the review:

> *"It feels like I might get a bit colored by getting the improvements from the LLM [...] I feel like maybe I could miss something else, because I would focus on those improvements a lot."* [P3]

While some view using AI tools as risky, not all participants saw the assistant as risky. Some viewed it as a low-stakes addition to the workflow, especially when **its use remained optional**, even if it is not always accurate:

> *"As long as there's an opt-out option, there's no real harm in it."* [P5]

> *"If we miss 10 [issues] today, we might miss two with a tool like this."* [P9]

The **role of trust** emerged as a key factor in shaping how participants viewed the tool. A few emphasized that for the tool to be useful or even used in general, it must be trusted, but not blindly. Misplaced trust in the tool could lead to wasted effort if the tool isn't accurate:

> *"So I think that worked really well, especially with larger [PRs] [...] you can at least use it if you trust it."* [P7]

> *"I could go on for hours, just to realize I can never do this. Then I've just lost a few hours trying to pursue something that wasn't possible."* [P5]

2) Efficiency and Thoroughness: A strong theme was the AI's impact on code review efficiency and the thoroughness or quality of reviews. Developers reported that the AI assistant could **speed up the review process, reduce reviewers' workload on tedious tasks, and potentially catch more issues**, although some participants remarked that it **sometimes focuses on low-priority findings, introducing noise**.

Beyond efficiency, participants also pointed to improvements in **review quality**. Some noted that the assistant could identify issues that might otherwise go unnoticed:

> *"There are probably findings you get in the report that you don't find when you do it manually."* [P10]

Additionally, interviewers felt that the assistant would be particularly helpful on **large pull requests** where it would be difficult for a human reviewer to catch everything:

> *"It's taken someone two weeks to write it [...] giving it 15 minutes, you won't have a chance to understand it, at least not well enough to find the hard stuff. I would imagine that the tool would actually raise a flag for a potential deadlock or race condition as well."* [P7]

For some participants, the assistant reduced the effort involved in reviewing by **removing the need to search through the codebase or external documentation**:

> *"But I really like the integration with the requirements part, because if I open up [a PR] and I don't know what it's about. [...] The first thing I do every time is I open the [Jira] ticket anyway, because I need to see what is supposed to have been achieved here. So I think that's a really nice functionality to have"* [P3]

However, some participants noted that the assistant occasionally surfaced low-priority or unclear findings. This was also observed during review sessions, where participants sometimes expressed **difficulty distinguishing important findings from minor ones, especially in lengthy summaries generated by the assistant**.

3) Design Expectations and Limitations: Many participants shared expectations about how an AI assistant for code review should behave and be integrated. A recurring theme was the desire for **seamless integration into existing workflows and tools**. Rather than switching to a new interface, several participants expressed that it would be preferable to access the assistant directly from familiar environments like GitHub, Slack, or their IDEs. As one participant put it:

> *"I think most of the developers don't want to use something new, like a [new] UI, but rather have an integration to what exists."* [P11]

Building on this, another participant described a preference for having the assistant's comments embedded directly into GitHub's interface, with expandable in-line comment boxes, while also having the ability to further ask questions in a chat

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restriction

interface. Yet another participant described how having it in the pipeline through a Slack bot could be useful.

> *"Maybe a Slack auto bot could even be triggered on each message [...] and a full review could be dropped as a message under that thread."* *[P11]*

Beyond integration, participants also critiqued how the LLM assistant's output was presented. Several interviewees felt that the **LLM feedback was overly long or difficult to scan**. One participant noted:

> *"I mean, what's important is that it very clearly lists the file. I'd rather have it list the file and the line number and be very specific, in a short way."* *[P5]*

Response time was another point of friction. While some delays were tolerated, long wait times were cited as a major barrier to adopting the tool in real development workflows:

> *"I think the speed and accuracy are mainly what need to be improved. [...] I wouldn't use this if it took, I don't know, how many minutes it took for it to respond."* *[P3]*

One participant reflected on whether the quality of interaction also depended on their **own ability to ask good questions**:

> *"Maybe my way of asking questions was also wrong. I did not always feel like I got the response that I was asking for. So, I might need to learn how to be more detailed in my questions."* *[P10]*

Many limitations were traced to a **lack of access to broader context, such as architectural documentation, internal conventions, or metadata**:

> *"Ideally, you want to inject as much relevant information as possible [...] like the JIRA ticket, relevant [documentation] pages, the codebase itself, the README, and any similarly named repositories that might be connected to the same service."* *[P5]*

Some participants also highlighted that the **LLM-assistant's usefulness depended in part on how good the documentation and PR descriptions are to begin with**. During the testing, one reviewer reflected that the tool could help keep the documentation up-to-date by suggesting updates that align with PR changes.

4) Usage Contexts and Interaction Patterns: Participants expressed a range of preferences, strategies, and situational factors that shaped how they interacted with the AI assistant, often shaped by the review context. These patterns included both the predefined interaction modes: Mode A (Co-Reviewer) and Mode B (Interactive Assistant), as well as emergent workflows that blended or extended beyond them.

Many participants **found Mode A (Co-Reviewer) especially helpful for getting oriented in a pull request**. They described the high-level summaries and suggestions provided at the beginning of the review as useful for gaining quick context, particularly in unfamiliar or complex codebases:

> *"I prefer this one [Mode A] where you actually get the overview directly [...] it had a lot of good pointers, that it already found."* *[P12]*

> *"The first engine [Mode A] that gave me a breakdown of everything [...] that was quite clever, and I would gladly use that."* *[P7]*

Mode A was also described as particularly **useful for low-risk PRs**:

> *"Let's say the change is relatively small and it's not causing any risk, then I would definitely go with the first one [Mode A], where I let AI do most of the work."* *[P11]*

Many participants saw the assistant, especially mode A, as **valuable for newcomers**:

> *"I think if I were in a new team, and I am unsure what is happening, then it could be really good to start with a summary."* *[P8]*

> *"Yeah, if you can write questions like 'What is this?' or 'What is this really about?', it could also be a very good tool to get to know the codebase and to learn as a new guy."* *[P9]*

Additionally, some saw mode A as **useful in teams where code review standards are lower or in teams that prefer other methods for reviewing code, such as pair programming**. In such cases, the assistant would serve as a fallback mechanism.

> *"I think it would be great for those who usually just skim through and say, 'It looks good to me'. [...] I think the biggest effect would be for those developers, I guess, and those teams."* *[P5]*

Mode B was less preferred in general, but some participants remarked that in cases where they are **already familiar with the codebase or if they wanted to maintain full control over the review process, they would prefer Mode B**:

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio

> *"But if it's in some codebase I already know, some codebase where we have a lot of experience and have worked in it a lot. It could probably be nice to have [Mode B]."* *[P8]*

In some cases, participants preferred a **combination of both modes** or expressed that the preferred interaction mode depended on the situation:

> *"I think I'm 50/50 [...] both are useful. One is on demand, the other one is on its own."* *[P1]*

Several participants **proposed additional usage patterns** that were not strictly defined by the study design. For instance, some saw mode A as useful for the author before submitting the PR, rather than during review:

> *"I feel it might not be as much of a review help. I think it might be a pre-review help."* *[P10]*

Others **proposed an alternative interaction mode** where engineers conducted a human-led review first and then used the assistant to validate or catch anything they might have missed:

> *"You start out with it just to sum up what the code is doing. Then I look for issues, and then I can ask, 'Are there any further issues?"* *[P3]*

Participants also frequently noted that Mode A was especially **helpful for large PRs**:

> *"Especially for large PRs, it's nice to get the breakdown on what's happening [...] because usually, you always have to do that sort of manually anyway."* *[P3]*

However, there was also some uncertainty about how **effective the assistant would be at scale**. One participant expressed concerns about the assistant's ability to handle very large codebases or complex business logic:

> *"I think it's going to be a bottleneck for such things, because there will be so many moving parts in it, so much business logic going around."* *[P1]*

VI. Implications

Our study yields several practical implications for integrating LLMs into code review workflows. First, AI assistance should be embedded within developers' existing tools (such as GitHub, GitLab, IDEs, or Slack) to minimize friction and support natural adoption. Second, output must be concise, well-structured, and actionable, prioritizing critical findings with precise references to affected files and lines. Fast response times are essential to preserve reviewer flow, although more comprehensive agentic reviews may be acceptable when integrated into automated pipelines. Supporting both proactive (AI-led summaries) and reactive (on-demand Q&A) modes of interaction is key, with a general developer preference for AI-led summaries in large or unfamiliar pull requests. For meaningful support, the assistant must be context-aware and have access to relevant information such as code diffs, source files, and requirement documents. While our tool addressed this via a retrieval-augmented setup, participants still highlighted the need for deeper contextual integration. Finally, an LLM-enabled assistant also shows promise as a pre-review aid, helping authors catch simple issues before submitting a pull request, thus improving code quality upstream in the development lifecycle.

References

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[3] T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love *et al.*, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.

[4] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[5] R. Tufano, A. Martin-Lopez, A. Tayeb, S. Haiduc, G. Bavota *et al.*, "Deep learning-based code reviews: A paradigm shift or a double-edged sword?" *arXiv preprint arXiv:2411.11401*, 2024.

[6] H. Y. Lin, P. Thongtanunam, C. Treude, and W. Charoenwet, "Improving automated code reviews: Learning from experience," in *International Conference on Mining Software Repositories (MSR)*. ACM, 4 2024, pp. 278–283.

[7] M. Vijayvergiya, M. Salawa, I. Budiselić, D. Zheng, P. Lamblin, M. Ivanković, J. Carin, M. Lewko, J. Andonov, G. Petrović, D. Tarlow, P. Maniatis, and R. Just, "AI-assisted assessment of coding practices in modern code review," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*. ACM, 2024, pp. 85–93.

[8] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, "AI-powered code review with LLMs: Early results," *arXiv preprint arXiv:2404.18496*, 2024.

[9] A. Alami and N. A. Ernst, "Human and machine: How software engineers perceive and engage with AI-assisted code reviews compared to their peers," *arXiv preprint arXiv:2501.02092*, 2025.

[10] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, "Towards automating code review activities," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 163–174.

[11] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.

[12] V. Braun and V. C. and, "Using thematic analysis in psychology," *Qualitative Research in Psychology*, vol. 3, no. 2, pp. 77–101, 2006.

[13] O. A. Adeoye-Olatunde and N. L. Olenik, "Research and scholarly methods: Semi-structured interviews," *JACCP*, vol. 4, no. 10, pp. 1358–1367, 2021.

[14] I. Etikan, S. A. Musa, R. S. Alkassim *et al.*, "Comparison of convenience sampling and purposive sampling," *American journal of theoretical and applied statistics*, vol. 5, no. 1, pp. 1–4, 2016.

[15] G. Guest, A. Bunce, and L. Johnson, "How many interviews are enough?: An experiment with data saturation and variability," *Field Methods*, vol. 18, no. 1, pp. 59–82, 2006.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:12:21 UTC from IEEE Xplore. Restrictio